

Универзитет у Београду
Електротехнички факултет



МРЕЖНА КОМУНИКАЦИЈА И ГРАФИЧКИ КОРИСНИЧКИ
ИНТЕРФЕЈС ЗА ПРАЋЕЊЕ ПОТРОШЊЕ НАМЕНСКИХ
РАЧУНАРСКИХ СИСТЕМА

Дипломски рад

Ментор:

проф. др. Владимир Рајовић
ванредни професор

Кандидат:

Василије Чарапић
2022/0218

Београд, Септембар 2026.

Сажетак

У оквиру овог рада реализован је мрежни слој на ESP микроконтролеру за пренос мерних података и управљачких команди, као и интеграција одговарајуће софтверске апликације са графичким корисничким интерфејсом (GUI) за управљање преносом и визуелизацију напона и струје. STM микроконтролер, са софтверском подршком реализованом у оквиру FreeRTOS оперативног система за рад у реалном времену, служи као извор података, при чему су током тестирања коришћени псеудослучајно генерисани одбирци напона и струје. Поред тога, имплементирана је подршка за INA234 сензор који комуницира преко I2C магистрале, која у оквиру овог рада није функционално верификована са додатним оптерећењем (повезаним оптерећењем на сензору).

ESP микроконтролер има улогу преноса података између STM микроконтролера и GUI апликације: путем UART серијске везе прима податке и команде од STM микроконтролера, док са GUI апликацијом комуницира преко TCP сервера (команде за управљање системом) и UDP протокола (пренос одбирака). У раду је детаљно описан начин на који GUI апликација омогућава конфигурацију система, приказ напона, струје и кумулативне потрошње у реалном времену, као и чување профила потрошње и увид у статистику пријема података. Тестирањем у свим предвиђеним условима рада потврђена је поузданост преноса и није забележен ниједан изгубљени пакет.

Кључне речи: ESP32, TCP/UDP, графички кориснички интерфејс, анализа потрошње енергије, пренос података, FreeRTOS, INA234

Садржај

1	Увод	1
2	Теоријски увод	2
2.1	Системи за мерење потрошње	2
2.2	Улога графичког интерфејса у анализи потрошње енергије	2
2.3	Комуникациони протоколи	2
2.3.1	I2C	3
2.3.2	UART	4
2.3.3	TCP и UDP	4
2.4	Прослеђивање порука преко мреже и машина стања	6
2.5	Окружење за развој графичких апликација	6
3	Хардвер система	7
3.1	STM микроконтролер	7
3.2	ESP микроконтролер	7
3.3	Повезивање и програмирање система	8
4	Софтвер система	10
4.1	Софтвер STM микроконтролера	10
4.1.1	FreeRTOS архитектура на STM (таскови, редови, нотификације)	10
4.1.2	UART слање порука (TxMsg_t, редови, мутекс)	11
4.1.3	Извор мерних података: случајни генератор и INA234	12
4.2	Софтвер ESP микроконтролера	13
4.2.1	Организација и главна петља	13
4.2.2	Пријем података са STM-а	13
4.2.3	Мрежни слој и сервери	15
4.2.4	Токови података и формат пакета	17
5	Графички кориснички интерфејс	19
5.1	Окружење и покретање	19
5.2	Кориснички интерфејс и функционалности	22
5.2.1	Секција за додавање уређаја	22
5.2.2	Системска секција	22
5.2.3	Чување резултата	23
5.2.4	Контрола и напредне алатке	23
5.2.5	Дневник догађаја	25
5.3	Приказ и тумачење мерних података	26
6	Резултати тестирања	28
6.1	Приказ резултата	28
6.2	Статистика пријема	30
7	Закључак	31
8	Литература	32
	Списак скраћеница	33
	Списак слика	34
	Списак табела	35

1 Увод

Како се све више наменских рачунарских система развија, потрошња електричне енергије постаје један од кључних параметара при пројектовању електронских уређаја. Прецизно мерење и праћење потрошње омогућава рано откривање неефикасности у раду хардвера и софтвера, оптимизацију трајања батерије и валидацију енергетских система пре њиховог пуштања у производњу. Комерцијална решења за ову намену често су затвореног типа, скупа и тешко прилагодљива специфичним потребама конкретног пројекта, што отвара простор за приступачне алате отвореног кода.

Проблем којим се овај рад бави јесте пренос мерних података и управљачких команди између микроконтролера који врши мерење и рачунара на коме се подаци прате. Овај пренос мора истовремено да задовољи два захтева: поуздано преношење управљачких команди, уз брз пренос мерних величина у реалном времену. Пошто микроконтролер који врши мерење (STM) нема сопствену бежичну везу, потребан је посредник који преузима улогу прослеђивања пристиглих команди на одговарајућу дестинацију, као и апликација која кориснику омогућава да ове податке прати и да њима управља.

Циљ овог рада јесте реализација мрежног слоја на ESP микроконтролеру, који путем UART везе прима податке и команде од STM-а и прослеђује их ка рачунару преко TCP (команде) и UDP (мерни одбирци) протокола, као и интеграција са постојећом *Open-EPT GUI* апликацијом и приказ њеног коришћења: повезивање уређаја, конфигурација мерења, визуелизација напона, струје и потрошње у реалном времену, чување профила потрошње и увид у статистику пријема. Као извор података током тестирања користе се генерисане вредности познатог опсега, што омогућава проверу целокупног ланца преноса, паковања и скалирања независно од физичког сензора. Подршка за сензор INA234 имплементирана је као допуна, али није тестирана са реалним оптерећењем.

Систем се састоји од три целине: STM микроконтролера, који периодично прикупља вредности напона и струје и шаље их ка ESP-у, ESP микроконтролера, и GUI апликације на рачунару. Део система који се односи на STM (реализован у оквиру FreeRTOS оперативног система) описан је у мери која је неопходна за разумевање формата података који се шаљу ка ESP-у, док је GUI апликација отвореног кода коришћена као готов алат.

2 Теоријски увод

У овом поглављу дат је преглед теоријских концепата на којима почива реализација система: слична решења за праћење напона и струје у реалном времену, комуникациони протоколи коришћени између компоненти система (UART, I2C, TCP, UDP), као и окружење за развој графичких апликација.

2.1 Системи за мерење потрошње

Комерцијална решења за анализу потрошње енергије (нпр. Monsoon Power Monitor, Nordic Power Profiler Kit) по правилу представљају затворене, наменске уређаје који нуде високу прецизност, али ограничену могућност прилагођавања сопственим протоколима и форматима података, док истовремено захтевају значајна финансијска средства. Као одговор на ово ограничење, у оквиру академске заједнице развијају се решења отвореног кода, попут платформе *Open-EPT* [4], на којој се и заснива овај рад, а која раздваја улогу прикупљања података од улоге визуелизације и управљања. Раздвајање временски критичног прикупљања података од његове визуелизације уобичајен је приступ и у индустријским мерним инструментима (осцилоскопима, логичким анализаторима), где се наменски систем ослања на оперативни систем за рад у реалном времену, док се приказ и обрада података препуштају апликацији на рачунару вишег нивоа апстракције [10, 11].

2.2 Улога графичког интерфејса у анализи потрошње енергије

Мерење потрошње струје има смисла само ако се из прикупљених података могу извући конкретни закључци. Пошто сензор при периоду одабирања од 1 ms генерише велики број података у секунди, графички кориснички интерфејс (GUI) је неопходан за праћење рада система. Поред овога, кроз графички кориснички интерфејс се подешавају параметри мерења без измене *firmware*-а, прати евентуални губитак пакета у мрежи и архивирају профили ради упоређивања ефеката оптимизације.

За разлику од комерцијалних алата који су углавном везани за затворен софтвер, *Open-EPT GUI* [1] је изабран јер је отвореног кода и могућности извршавања на linux и windows оперативном систему.

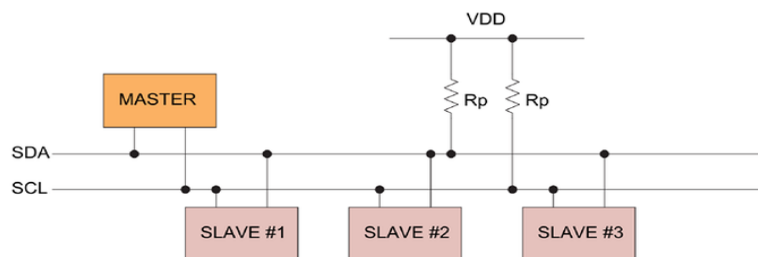
2.3 Комуникациони протоколи

Систем коришћен у овом дипломском раду интегрише више различитих комуникационих интерфејса, при чему сваки повезује другачији пар компоненти: сензор и STM микроконтролер повезани су I2C магистралом, два микроконтролера међусобно UART серијском везом, док се комуникација између уређаја и рачунара остварује преко IP мреже, коришћењем TCP и UDP протокола.

2.3.1 I2C

I2C (*Inter-Integrated Circuit*) је синхрони серијски протокол намењен комуникацији на кратким растојањима, унутар исте плочице, између микроконтролера и периферија (сензора, EEPROM меморија, сатова реалног времена и сл.) [5]. Одликује се тиме што за повезивање већег броја уређаја користи само две линије: **SDA** (подаци) и **SCL** (такт) које деле сви уређаји на магистрали.

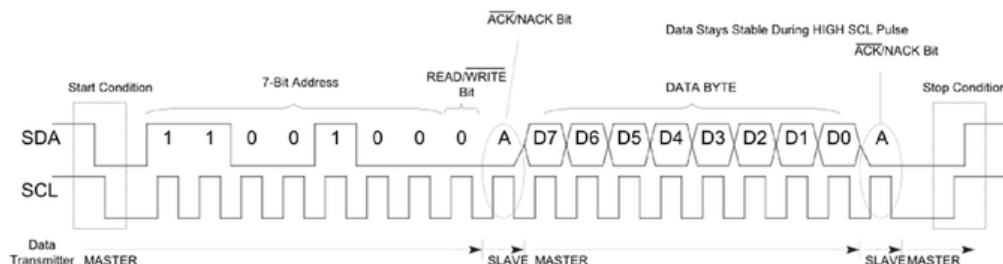
Обе линије су реализоване као *open-drain* излази са спољашњим *pull-up* отпорницима, па је магистрала у мировању на високом логичком нивоу, а сваки уређај може да је само повуче на низак ниво.



Слика 1: Шема повезивања на I2C магистрали

Комуникација се заснива на *master-slave* архитектури. *Master* (у овом систему STM микроконтролер) генерише такт на линији **SCL** и иницира сваку трансакцију, док *slave* уређаји само одговарају на упите. Сваки *slave* има јединствену 7-битну адресу преко које га master адресира, па се на исту магистралу може прикључити више уређаја. На магистрали **I2C3** налази се сензор **INA234** са адресом **0x40**.

Свака трансакција почиње *START* условом (**SDA** прелази са високог на низак ниво док је **SCL** на високом), након чега master шаље адресни бајт (7-битна адреса *slave* уређаја и бит смера преноса). Уређај чија се адреса поклапа потврђује пријем (*ACK*) повлачењем **SDA** на низак ниво, затим следе бајтови података са потврдама, а трансакција се завршава *STOP* условом (**SDA** прелази са ниског на високи ниво док је **SCL** на високом). Подаци смеју да се мењају само док је **SCL** на ниском нивоу.

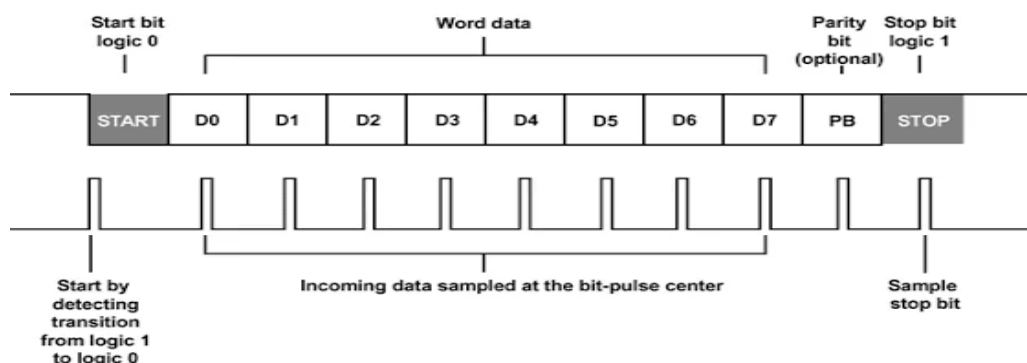


Слика 2: I2C магистрала и структура једне успешне трансакције

2.3.2 UART

UART (*Universal Asynchronous Receiver-Transmitter*) је асинхрони серијски протокол за комуникацију између два уређаја преко две линије: **TX** (слање) и **RX** (пријем). Линије се повезују унакрсно, тако да **TX** једног уређаја иде на **RX** другог и обрнуто, уз заједничку масу. Пошто свака страна има засебну линију за слање, оба уређаја могу истовремено да шаљу и примају податке.

Протокол не користи заједнички сигнал такта. Уместо тога, обе стране морају бити унапред подешене на исту брзину преноса (*baud rate*), на основу које пријемник самостално издваја пристигле бајтове. Подаци се шаљу у оквирима: линија је у миновању на високом нивоу, а почетак оквира означава *start* бит (низак ниво). Након њега следе битови података (најчешће осам, почевши од најнижег), по потреби бит парности и један или два *stop* бита (висок ниво). Пријемник детектује силазну ивицу *start* бита и затим читава сваки бит на средини његовог трајања, због чега су дозвољена само мала одступања у брзини између две стране. При уобичајеном формату 8N1 сваки бајт података преноси се помоћу 10 бита. Структура једног оквира приказана је на слици 3.



Слика 3: Структура једног оквира UART пакета

За разлику од I2C-а, UART повезује тачно два уређаја и нема адресирање: све што једна страна пошаље стиже до друге. Због одсуства заједничког такта и линије за потврду пријема UART је једноставан за реализацију и погодан за комуникацију на кратком растојању између две плочице. Са овим протоколом могуће је детектовати грешке на нивоу једног бајта (грешка парности, грешка оквира, прекорачење пријемног бафера).

2.3.3 TCP и UDP

Комуникација између уређаја и рачунара одвија се преко скупа протокола познатог као TCP/IP стек. Стек је организован у слојеве, при чему сваки слој користи

услуге слоја испод себе и пружа услуге слоју изнад њега. На дну се налази слој везе (у овом систему Wi-Fi), изнад њега мрежни слој са протоколом IP који адресира уређаје и усмерава пакете кроз мрежу, затим транспортни слој (TCP и UDP) који омогућава комуникацију између апликација на два уређаја, а на врху апликативни слој. У OSI референтном моделу транспортни слој је четврти по реду, и њему припадају TCP и UDP. Апликације у оквиру транспортног слоја разликују се по броју порта (нпр. порт 8080 на коме ESP одржава TCP сервер), па пар IP адресе и порта јединствено одређује крајњу тачку комуникације. Архитектура поделе на слојеве приказана је на слици 4.



Слика 4: Положај TCP и UDP протокола у мрежним слојевима

TCP (*Transmission Control Protocol*) [15] је протокол који гарантује испоруку сваког бајта, у редоследу у ком је послат, уз потврде и поновно слање изгубљених сегмената. Цена тога је додатно кашњење и рад са потврдама, али је за управљачке команде то прихватљиво, а исправност је неопходна, јер изгубљена команда (нпр. старт аквизиције) не сме да прође непримећено.

UDP (*User Datagram Protocol*) [14] је протокол без успостављања везе: пошилац једноставно шаље независне датаграме, без потврда, без поновног слања и без гаранције редоследа. Због малог оптерећења и одсуства чекања на потврде, погодан је за проток мерних одбирака у реалном времену, где је важније да нови подаци стижу брзо него да се старији поново шаљу. Пошто UDP сам не пријављује губитке, у формат пакета уграђен је бројач пакета на основу ког пријемна страна може да открије да ли је неки пакет изгубљен.

Величина UDP датаграма при томе мора бити ограничена испод **MTU** (*Maximum Transmission Unit*) вредности мреже (типично 1500 бајтова за Ethernet/WiFi), односно највеће величине оквира коју мрежни слој може да пренесе без дељења. Уколико датаграм премаши ову границу, долази до *IP фрагментације* — дељења датаграма на више IP пакета на страни пошилаоца и њиховог поновног састављања на пријемној

страни — што повећава кашњење и вероватноћу губитка целог датаграма уколико се изгуби само један од фрагмената.

Особина	TCP	UDP
Веза	успоставља се пре преноса	без везе
Поузданост	гарантована испорука и редослед	без гаранције
Кашњење	веће (потврде, поновно слање)	мање
Употреба у систему	команде GUI $\leftrightarrow$ ESP	пренос одбирака ESP $\rightarrow$ GUI

Табела 1: Поређење TCP и UDP протокола и њихова улога у систему

2.4 Прослеђивање порука преко мреже и машина стања

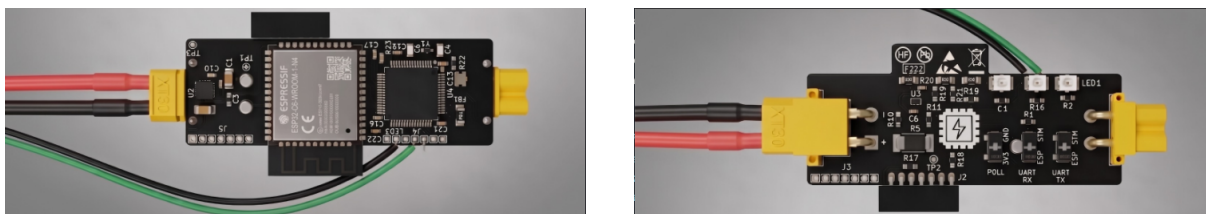
Пошто STM микроконтролер у овом систему нема сопствену бежичну везу (WiFi), на плочици постоји ESP микроконтролер који преузима улогу прослеђивања порука између две стране (мрежног моста): прима бајтове са серијске везе и прослеђује их даље преко мреже, и обрнуто. Микроконтролер притом мора да разликује врсте података које стижу на исту серијску линију — текстуалне одговоре и бинарне мерне пакете. Овај проблем се природно решава *машином стања* (*finite state machine*) — моделом рада у коме систем у сваком тренутку има тачно једно активно стање и на основу текућег стања и пристиглог улаза (нпр. следећег бајта) прелази у ново стање или предузима одговарајућу акцију. Оваква обрада прима улаз бајт по бајт, не блокира извршавање програма и не захтева да цела порука буде присутна у меморији пре почетка обраде.

2.5 Окружење за развој графичких апликација

Орен-ЕРТ GUI апликација реализована је у радном оквиру *Qt*, међуплатформском оквиру за C++. Одазивност корисничког интерфејса обезбеђује се раздвајањем главне нити извршавања од радних нити задужених за пријем мрежних података. Резултат обраде у радној нити прослеђује се главној нити асинхроно, механизмом сигнала и слотова [16], чиме се избегава блокирајуће чекање главне нити на мрежне операције и чиме график остаје употребљив чак и при великом протоку података. Развојно окружење *Qt Creator* коришћено је као интегрисани алат за превођење и покретање апликације.

3 Хардвер система

Систем на коме је реализован овај рад је развојни систем Open-EPT (CompactLP), који на једној плочи обједињује све делове потребне за мерење и пренос података: STM микроконтролер као главну управљачку јединицу, ESP микроконтролер као комуникациони посредник између STM-а и GUI апликације, као и сензор INA234 за мерење напона и струје. Плоча је приказана на слици 5.



Слика 5: Open-EPT плоча са STM и ESP микроконтролером

Компоненте на плочи међусобно су повезане на три начина. STM микроконтролер са сензором INA234 комуницира преко I2C магистрале, са ESP микроконтролером преко UART серијске везе, док ESP са рачунаром комуницира бежично, преко Wi-Fi мреже, коришћењем TCP и UDP протокола. Оваква подела омогућава да се временски критично прикупљање података обавља на STM микроконтролеру, а да се пренос ка рачунару препусти ESP-у.

3.1 STM микроконтролер

Улогу главне управљачке јединице има микроконтролер STM32L476RG. Његови меморијски ресурси (Flash и RAM) омогућавају стабилан рад FreeRTOS оперативног система, уз истовремено управљање више FreeRTOS структура као што су таскови, редови за поруке и мутекси. Поред тога, интегрише хардверске периферије које се користе у овом систему: I2C магистралу за комуникацију са сензором, UART интерфејсе за серијску комуникацију са ESP модулом (`huart4`) и са рачунаром (`huart2`), као и хардверске тајмере који омогућавају прецизно периодично читавање канала и генерисање прекида.

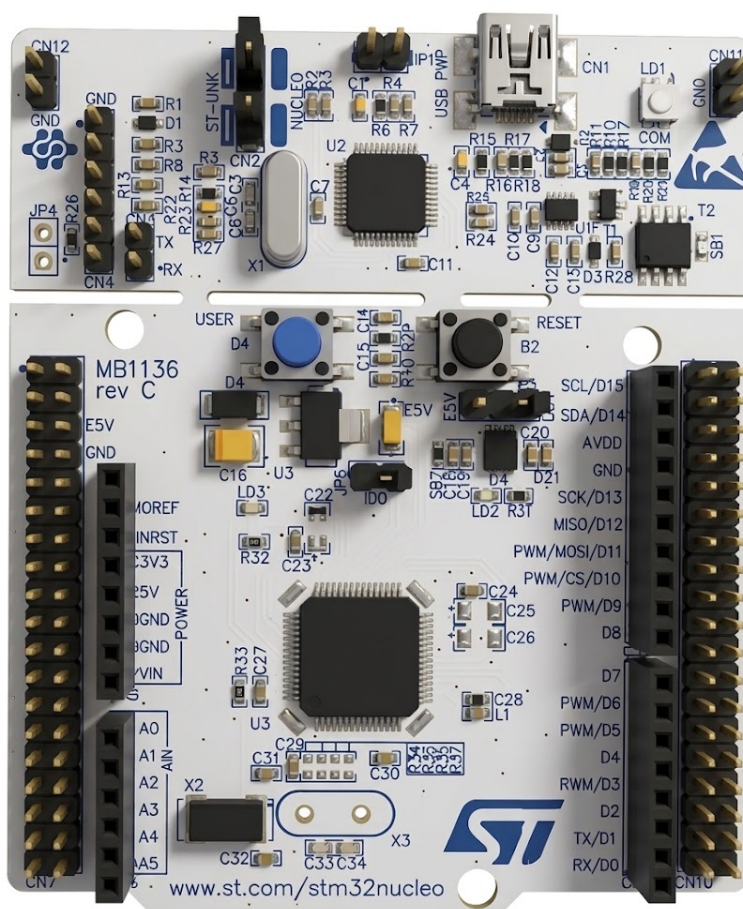
3.2 ESP микроконтролер

За реализацију бежичне везе са рачунаром користи се микроконтролер ESP32-C6. Захваљујући интегрисаном Wi-Fi модулу и подршци за рад у Station или Access Point режиму, он омогућава бежични пренос података преко локалне мреже. На њему је имплементирана подршка за TCP и UDP мрежне протоколе, где се TCP користи за поуздан пренос управљачких команди, а UDP за брз пренос измерених вредности

у реалном времену. Поред мрежне функције, ESP преко серијске UART везе прима податке и команде од STM-а и прослеђује их даље кроз мрежу, и обрнуто. Софтвер ESP микроконтролера описан је у наредном поглављу.

3.3 Повезивање и програмирање система

За програмирање и напајање Ореп-ЕРТ плоче користе се NUCLEO-L476RG плочица (слика 6) и USB to TTL програматор (слика 7). NUCLEO плочица служи за програмирање и дебаговање STM микроконтролера, док се USB to TTL програматор користи за програмирање ESP32-C6 модула.

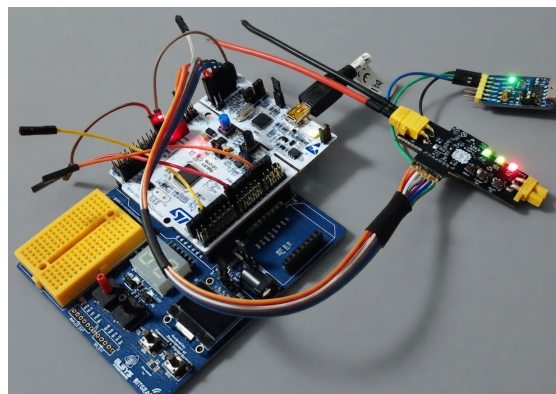


Слика 6: STM NUCLEO L476RG

Одговарајући пинови за програмирање, дебаговање и напајање система повезани су на основу поступка описаног у поглављу 7.4.5 корисничког упутства за NUCLEO-L476RG [13] и шематског приказа Ореп-ЕРТ плоче. Повезани систем приказан је на слици 8.



Слика 7: USB To TTL програматор



Слика 8: Повезивање система са Ореп-ЕРТ плочом

Програмирање система обавља се у неколико корака. Најпре се са репозиторијума CLP преузима *firmware*. Затим се пројекат отвара у развојном окружењу STMCubeIDE и CLP се поставља као активан пројекат (*active project*), након чега се он покреће, чиме се код флешује и извршава на STM микроконтролеру. Док STM код ради, покреће се *build* и *run* ESP пројекта преко USBtoTTL адаптера. Када се ESP код спусти на плочу, врши се ручно искључивање и поновно укључивање система, како би модул изашао из *boot* режима и започео извршавање пренетог кода. По покретању ESP32-C6 модула покреће се одговарајући код на STM микроконтролеру.

Овакав поступак је неопходан јер *firmware* у CLP директоријуму садржи уграђени мост (*bridge*) преко кога је са STM микроконтролера могуће програмирати ESP32-C6 модул и пребацити га у *boot* режим. Одговарајући пинови налазе се унутар саме плоче, па ESP32-C6 модул није могуће програмирати екстерно, преко USB-а, на уобичајен начин.

4 Софтвер система

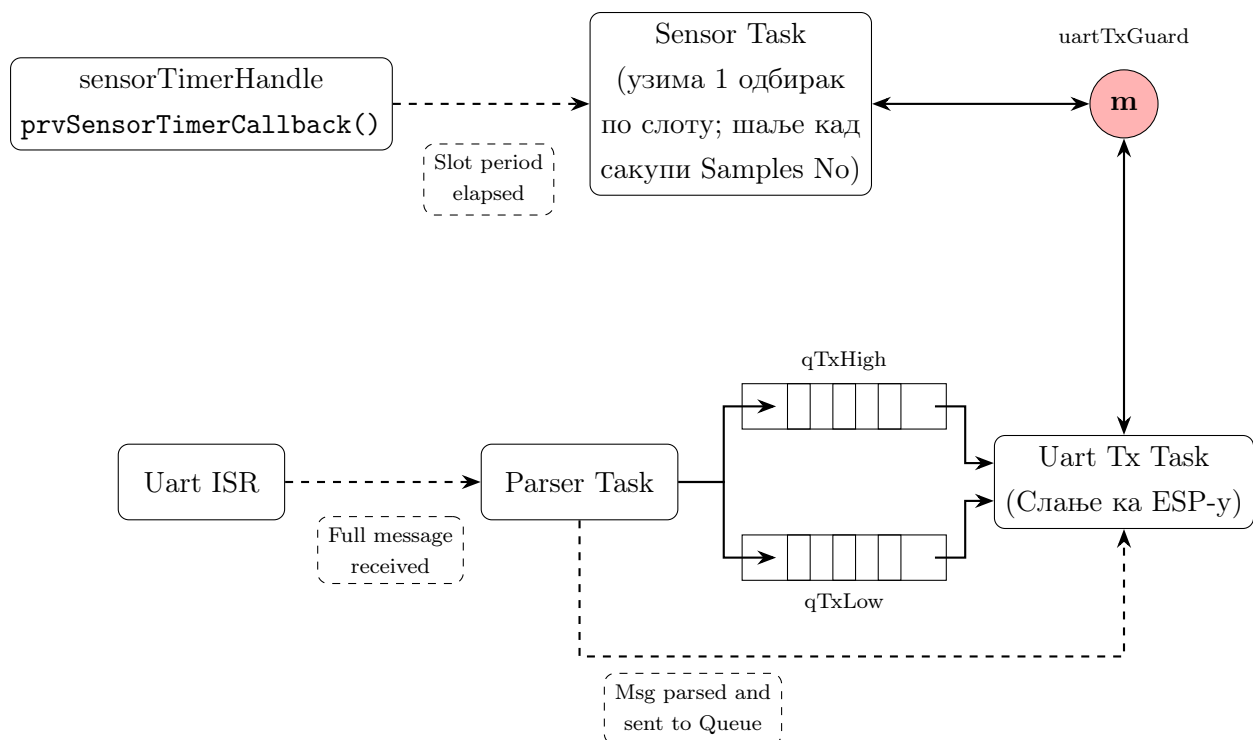
Ово поглавље прелази са теорије на конкретну реализацију система. Најпре је описан STM део, где мерни подаци настају, а затим и ESP део, преко кога ти подаци стижу до корисника. Сав софтвер система доступан је у репозиторијуму CompactLP који је вођен помоћу Git програма за контролу верзија.

4.1 Софтвер STM микроконтролера

Мерни подаци настају на STM страни, па је потребно укратко описати начин на који се они генеришу и шаљу, јер од тога зависи формат који ESP прима и прослеђује.

4.1.1 FreeRTOS архитектура на STM (таскови, редови, нотификације)

Дијаграм пројектованог система на STM страни приказан је на слици 9:



Слика 9: FreeRTOS архитектура система

Софтверска архитектура система на STM микроконтролеру реализована је у FreeRTOS оперативном систему за рад у реалном времену. Систем се састоји од три таска, софтверског тајмера, прекидне рутине (ISR), два реда са порукама и мутекса за заштиту дељеног хардверског ресурса.

Таскови имају следеће приоритете: **UartTxTask** (30) шаље све одговоре и поруке ка ESP-у и рачунару. **SensorTask** (20) периодично читава мерне канале и шаље би-

нарне поруке ка ESP-у. `ParserTask` (10) парсира примљене команде и рутира их ка одговарајућим руковаоцима.

`Uart` ISR нотификацијом (`xTaskNotifyFromISR`) поставља флег `MSG_RECEIVED_FLAG` чиме одблокира `ParserTask`. У зависности од критичности одговора, порука се смешта у `qTxHigh` (хитне потврде, контролне команде) или `qTxLow` (стандардни одговори) ред, након чега се `UartTxTask` нотификује одговарајућим флегом, где након примљене нотификације таск чита поруку и шаље је на UART.

Аквизиција почиње активирањем слања података: покреће се софтверски тајмер периода 1 ms који нотификује `SensorTask`. Таск броји протекле милисекунде за сваки ток података и узима одбирак за одређени ток података када се достигне вредност изведена из **Sampling Period**. По прикупљању **Samples No** одбирака формира се бинарни оквир и шаље ка ESP-у. Тако 1 ms представља доњу границу резолуције одабирања, а стварна брзина се независно усклађује по току података. Када аквизиција није активна, тајмер је заустављен и таск блокиран.

Приступ дељеном UART ресурсу (`SensorTask` и `UartTxTask`) заштићен је мутексом `uartTxGuard`.

4.1.2 UART слање порука (`TxMsg_t`, редови, мутекс)

STM комуницира преко два UART-а истовремено: `huart2` иде ка PC-ју, док `huart4` иде ка ESP-у. Пријем података је вођен прекидима, а након детекције комплетне поруке, парсер преузима садржај и прослеђује га на даљу обраду.

За слање података не користи се пристигла порука директно, већ је порука обавијена у структуру `TxMsg_t` и та структура је оно што представља нашу поруку. Садржај структуре приказан је на слици 10.

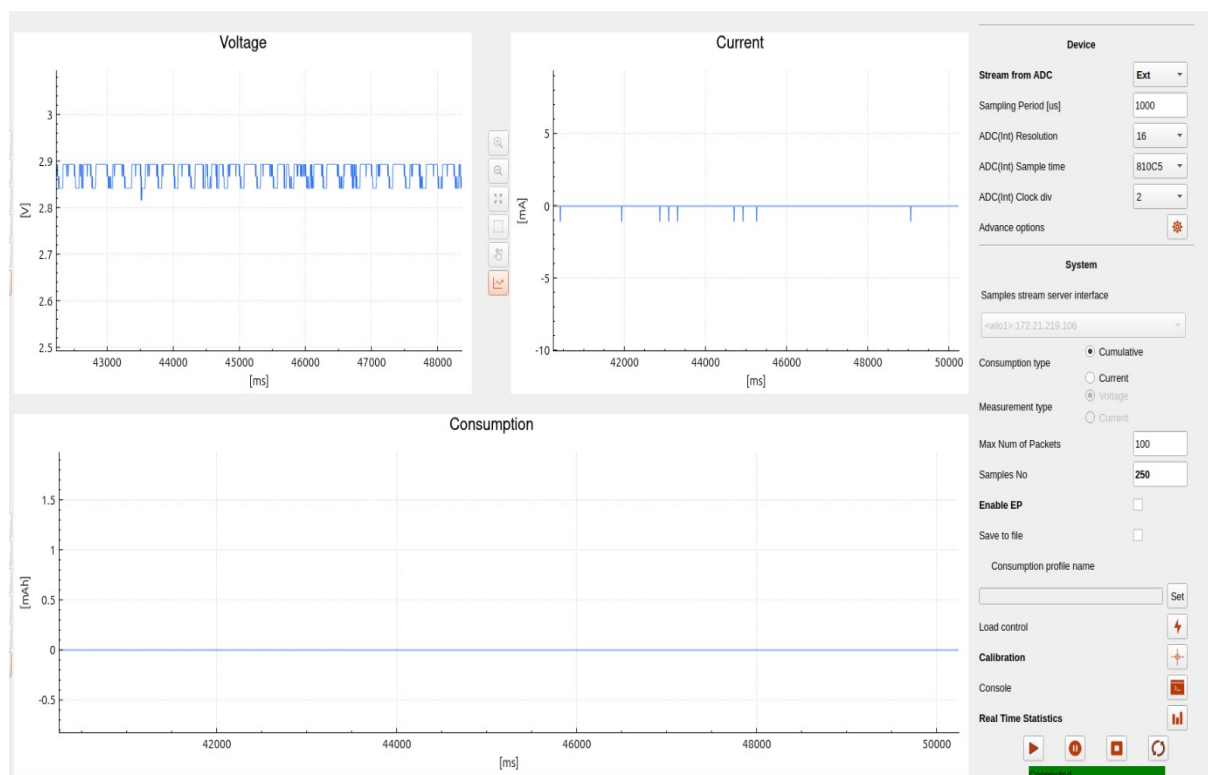
```
typedef struct {
    uint8_t data[150]; /*!< NUL-terminated message payload. */
    uint16_t len;      /*!< Payload length in bytes (excluding the NUL). */
    TxDest_t dest;     /*!< Resolved destination(s) for this message. */
} TxMsg_t;
```

Слика 10: Садржај `TxMsg_t` структуре

Порука се затим смешта у један од два FreeRTOS реда са различитим приоритетом: `qTxHigh` (за податке високог приоритета) или `qTxLow` (за стандардне одговоре парсера). Таск за слање порука путем UART-а, када добије нотификацију, најпре празни ред високог приоритета, а затим, када је он испразњен, празни ред ниског приоритета. Слање порука обавља функција `prvUARTTX_Transmit()`, која је заштићена мутекс семафором `uartTxGuard` и прослеђује податке на одговарајући UART интерфејс (или на оба, уколико је одредиште `TX_DEST_BOTH`).

4.1.3 Извор мерних података: случајни генератор и INA234

У раним фазама развоја софтвера, како би се систематично приступило проблему, мерења са сензора **INA234** су најпре занемарена, а пажња је усмерена на успостављање комуникације између STM микроконтролера и GUI апликације. У ту сврху **SensorTask** користи генератор случајних вредности, што омогућава проверу целог ланца преноса без физичког сензора. Када је потврђено да ланац преноса исправно ради, приступило се читању података са сензора **INA234** преко I2C магистрале (I2C3, адреса 0x40), који мери напон и струју преко шант отпорника од 40 mΩ, помоћу функција `HAL_I2C_Mem_Read()` и `HAL_I2C_Mem_Write()`. Вредности се кеширају на 10 ms ради смањења блокирајућих трансакција и скалирају у формат који GUI очекује, а у случају неуспешне иницијализације сензора систем се враћа на случајне вредности. Провера I2C комуникације извршена је без оптерећења (слика 11): напон варира око константне вредности, а струја је приближно нула јер ништа није повезано на крај сензора, чиме је потврђен комуникациони ланац до GUI-ја.



Слика 11: Очитавање напона, струје и потрошње са INA234 сензора без повезаног оптерећења

Очитавање сензора у кешу освежава се на 10 ms, док је најмањи период одабирања 1 ms, па више узастопних одбирака може потицати из истог очитавања. Усклађивање периода освежавања кеша остаје као даљи рад.

4.2 Софтвер ESP микроконтролера

Цео ESP firmware реализован је тако да омогући да се графички кориснички интерфејс повеже на уређај, шаље му команде и прима мерне податке у реалном времену. ESP преузима улогу мрежног моста: прима податке и команде са STM-а преко UART-а и прослеђује их даље преко мреже ка GUI-ју, и обрнуто. У наставку је описана организација firmware-а, начин на који се разликују текстуалне и бинарне поруке пристигле са STM-а, мрежни сервери преко којих се остварује веза са GUI апликацијом, као и формат пакета који стиже до GUI-ја.

4.2.1 Организација и главна петља

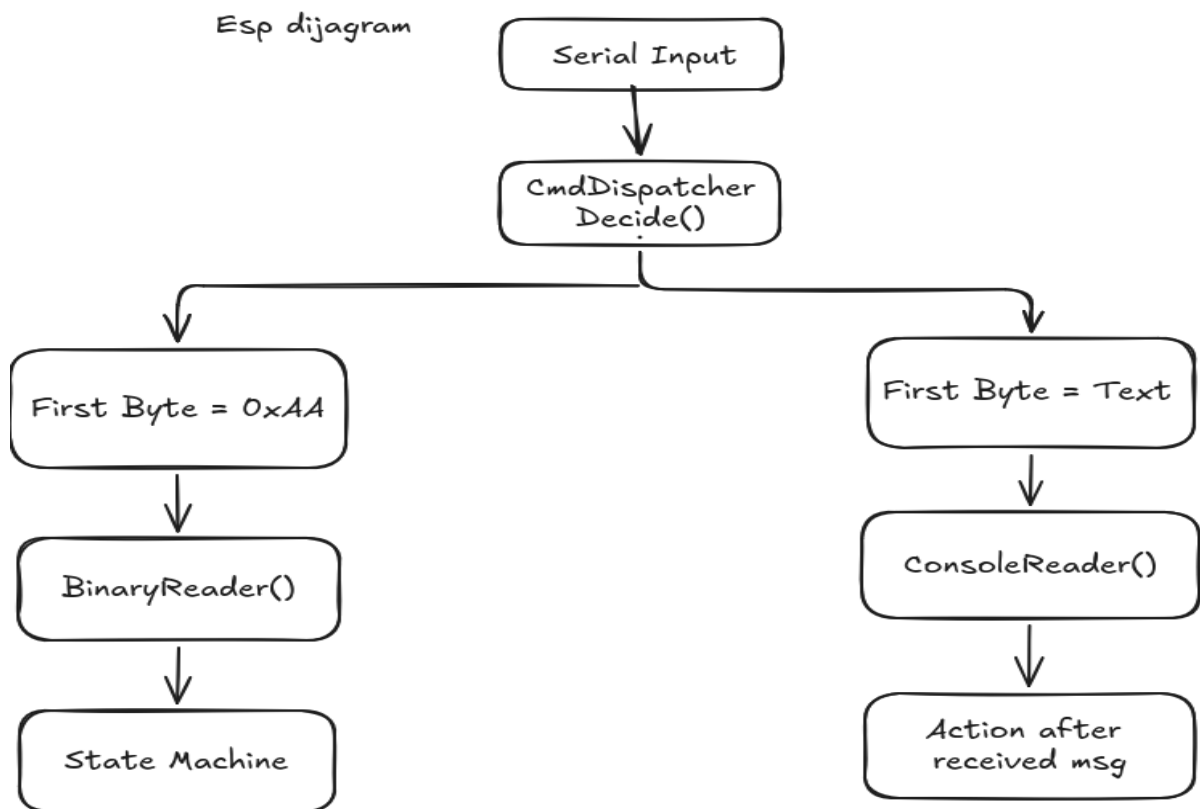
ESP firmware је написан у C++ програмском језику уз Arduino радни оквир. У `setup()` функцији подешава се серијска веза ка STM-у (921600 бауда), учитава под-разумевана мрежна конфигурација (режим рада, SSID, лозинка, портови) и покреће синхронизација времена (NTP) за време пристизања порука на веб серверу.

Функција `loop()` у сваком пролазу позива `netLayer.handleServers()` (одржава-ње сервера) и `dispatcher.decide()` (читање серијске везе), без блокирајућих опера-ција, чиме се обезбеђује да ниједна страна не чека дужи од потребног.

Firmware је организован око неколико целина: пријем и разврставање података са STM-а (`CmdDispatcher`), анализа и извршавање текстуалних команди (`CommandParser`), мрежни слој (`NetworkLayer` са компонентама `HttpServer`, `TcpServer`, `LinkManager` и `NetworkConfig`) и апстракција LED диоде и тастера (`Platform_t`) за локалне `show` команде.

4.2.2 Пријем података са STM-а

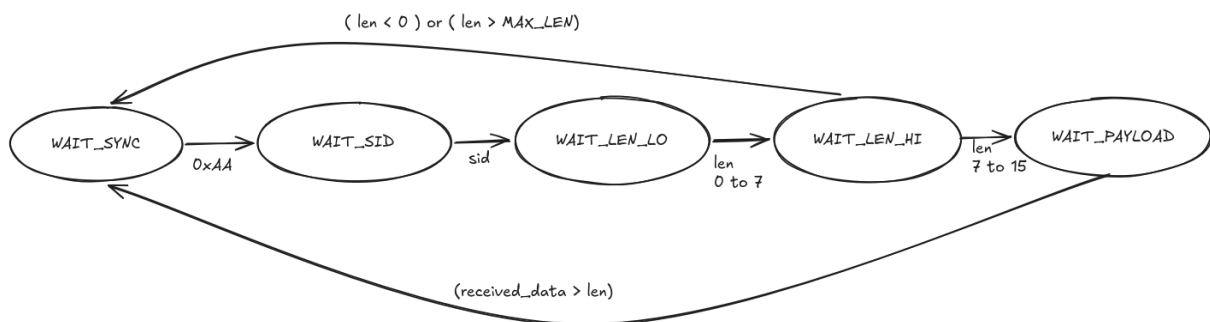
Са STM-а на исту серијску линију стижу две врсте података: текстуалне поруке и бинарни мерни пакети који почињу синхронизационим бајтом `0xAA`. Ове пристигле бајтове обрађује функција `CmdDispatcher::decide()`, машина стања која чита бајт по бајт и на основу првог бајта одлучује да ли прелази у ASCII или бинарни режим (слика 12).



Слика 12: Дијаграм диспачера команди на ESP-у

ASCII читач (`ConsoleReader`) гради линију бајт по бајт до знака краја линије (`\r`, `\n` или `\r\n`). Предугачке линије (преко 1000 знакова) се одбацују како не би блокирале пријем.

Бинарни читач (`BinaryFrameReader`) је машина стања са стањима `WAIT_SYNC` → `WAIT_SID` → `WAIT_LEN_LO/HI` → `WAIT_PAYLOAD`: чита идентификатор тока, дужину, па тачно толико бајтова садржаја. При неисправној дужини оквир се одбацује, а читач се увек враћа у `WAIT_SYNC`, чиме се систем аутоматски ресинхронизује и после губитка бајта на линији.



Слика 13: Машина стања бинарног читача (`BinaryFrameReader`)

Текстуалне поруке класификује `CommandParser::parseMessage()` према садржају прве речи, а извршава `CmdDispatcher::processLine()` (табела 2).

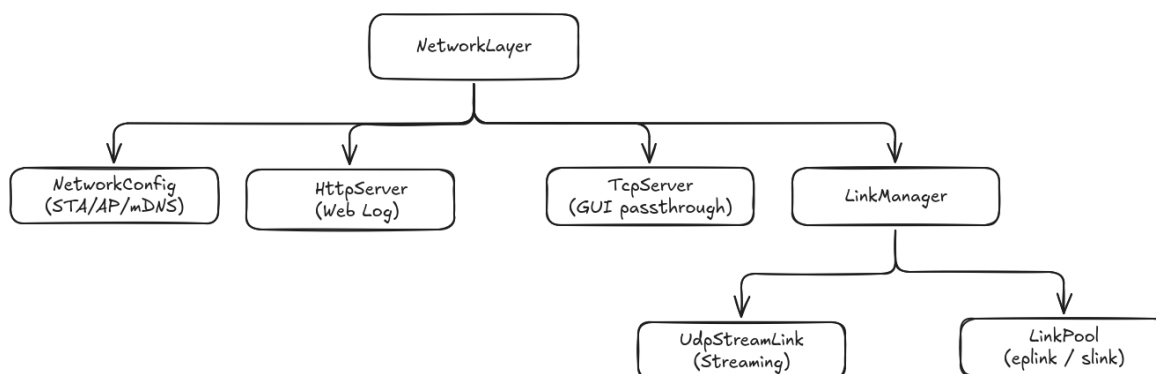
Префикс	Акција	Значење
ok, error	ACTION_GUI_FORWARD	одговор STM-а се прослеђује GUI-ју преко TCP-а
info:	ACTION_SERVER_RESTART	примењује се нова мрежна конфигурација и рестартују сервери
clear	ACTION_SERVER_CLEAR	брише се HTTP лог
echo:, check:	ACTION_LOG	порука се форматира и уписује у лог
show ...	ACTION_NONE	извршава се локално (LED/тастер преко Platform_t)
остало	ACTION_LOG	уписује се као неисправна порука

Табела 2: Акције `CommandParser`-а за текстуалне поруке са STM-а

Када бинарни оквир буде комплетан, ESP га прослеђује на UDP ток података који одговара његовом идентификатору тока (одељак 4.2.3).

4.2.3 Мрежни слој и сервери

Мрежни слој (`NetworkLayer`) обједињује три функционалности различите намене (слика 14): TCP сервер за управљачке команде са GUI-ја, UDP за пренос мерних одбирака и HTTP сервер за дијагностику.



Слика 14: Структура и компоненте мрежног слоја (`NetworkLayer`)

TCP сервер (`TcpServer`) прихвата једног активног GUI клијента. Сваки додатни покушај повезивања се одбацује, а на сокету се укључује опција `setNoDelay` ради мањег кашњења команди. Пристигле команде обрађује `forwardGuiToStm()`: команде везане за UDP токове података (`device stream/eplink/slink...`) се додатно тумаче преко `CommandParser::parseLinkCommand()` и извршавају преко `LinkManager`-а, док

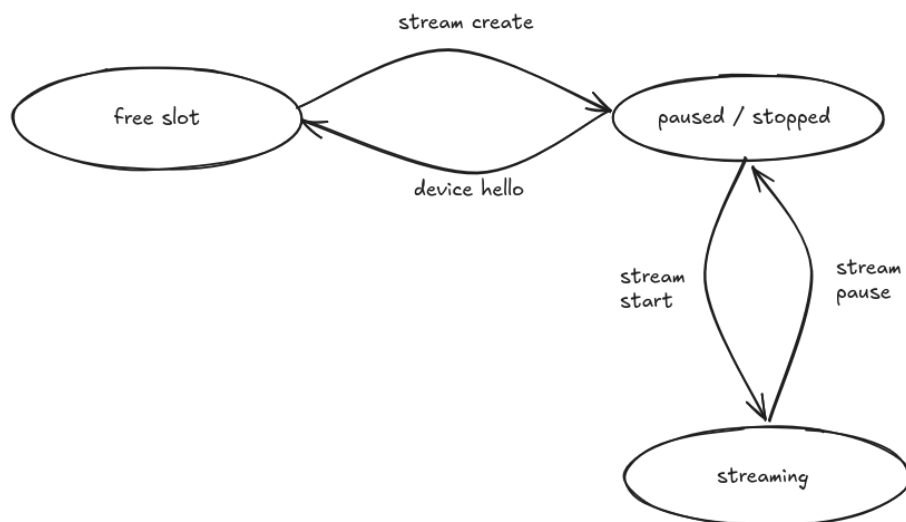
се свака команда непромењена прослеђује и STM-у преко UART-а, тако да ESP и STM држе усклађено стање.

Поред долазне везе са GUI-ја, **LinkManager** управља и са два додатна скупа TCP веза које STM може самостално да иницира преко ESP-а: **eplink** (до три везе) и **slink** (до две везе). Везе се отварају командом попут **eplink create -ip=... -port=...**, уз временско ограничење повезивања од 500 ms како би се избегло дуже блокирање функције **loop()**. Примљени подаци се са ових веза прослеђују ка STM-у преко UART-а.

Веза	Иницијатор	Макс. број	Повезивање	Обрада података
GUI	GUI (долазна веза на TcpServer)	1	додатни покушаји се одбацују; setNoDelay	команде се непромењене прослеђују STM-у преко UART-а; команде за UDP токове се извршавају преко LinkManager -а
eplink	STM, преко ESP-а	3	команда create ; тајмаут 500 ms	подаци се прослеђују STM-у преко UART-а
slink	STM, преко ESP-а	2	команда create ; тајмаут 500 ms	подаци се прослеђују STM-у преко UART-а

Табела 3: Преглед TCP веза које обрађује ESP

UDP пренос користи се за мерне одбирке због мањег кашњења у односу на TCP, уз секвенцијални бројач у заглављу који омогућава детекцију губитка пакета на пријемној страни. Систем подржава до три истовремена тока података, сваки идентификован јединственим бројем. Величина пакета је ограничена тако да остане испод MTU границе (1500 бајтова) и избегне IP фрагментацију. Идентификатор тока (*sid*) за нови UDP ток података ESP не прима од GUI-ја, већ га сам додељује: приликом **stream create** команде, **LinkManager** тражи први слободан слот у базену линкова и управо тај индекс постаје *sid*. Исти овај поступак дешава се и на STM страни, тако да обе стране потпуно независно долазе до истог *sid*-а. Максимална величина базена дефинисана је параметром **SSTREAM_CONNECTIONS_MAX_NUMBER** који у овом случају износи 3.



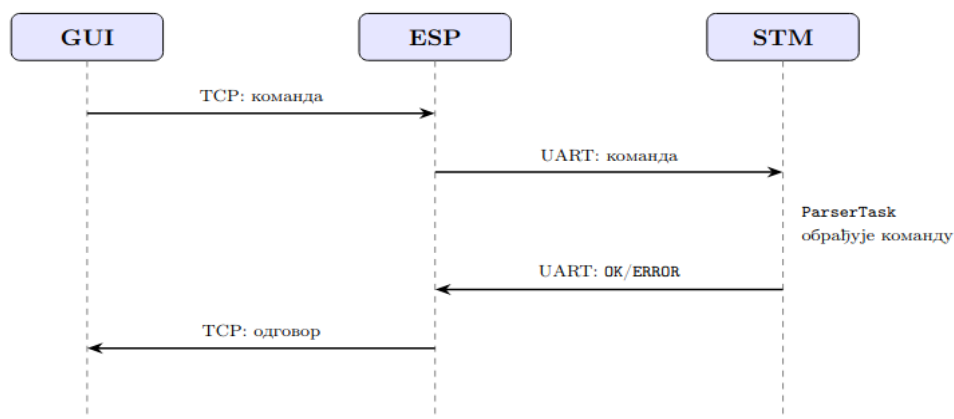
Слика 15: Стања једног UDP *stream* слота

Када GUI успостави нову TCP везу и пошаље почетну команду `device hello`, ESP потпуно ресетује базен UDP токова (ослобађа све слотове и *sid* вредности). Овим се стање на ESP страни усклађује са STM микроконтролером, који исту операцију извршава при пријему исте команде. Уколико GUI не пошаље `device hello` након прекида и поновног успостављања везе, алокација *sid* вредности на две стране може постати неконзистентна.

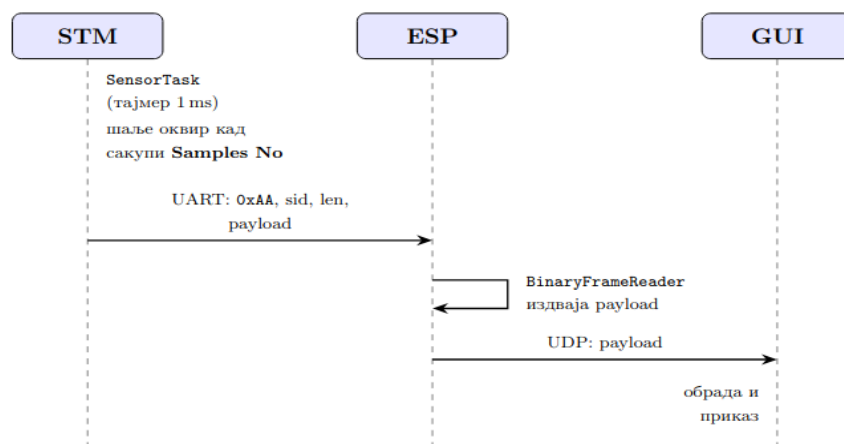
HTTP сервер (`HttpServer`) излаже руте `/test` и `/log` за преглед статуса и лога примљених порука, независно од GUI апликације, што омогућава проверу везе STM↔ESP.

4.2.4 Токови података и формат пакета

Слике 16 и 17 приказују токове података кроз систем у оба смера: пут једне команде од GUI-ја до STM-а и назад, и пут мерних одбирака од STM-а до GUI-ја.



Слика 16: Ток једне команде: GUI → TCP → ESP → UART → STM и назад



Слика 17: Ток мерних података: STM → UART → ESP → UDP → GUI

Структура бинарног пакета који STM шаље ка ESP-у приказана је на слици 18. Сама структура састоји се из заглавља (синхронизациони бајт, идентификатор тока, дужина садржаја) и корисног садржаја (бројач пакета, магични број и одбирци). ESP чита заглавље, а у UDP пакет прослеђује само корисни садржај. Скок у бројачу пакета на пријемној страни означава губитак пакета на путу.

Бајт 0	Бајт 1	Бајтови 2-3	Бајтови 4-7	Бајтови 8-11	Бајтови 12+
0xAA (Header)	Stream ID	Payload Length	Packet Counter	Magic Number	Payload Data

Слика 18: Структура бинарног пакета.

На основу брзине UART везе (921600 бауда) могуће је проценити расположиви проток података. При периоду одабирања од 1 ms и оквиру од 250 одбирака, подаци се шаљу на 250 ms при протоку од око 4 kB/s, што износи свега око 4,4 % капацитета везе – UART веза стога није ограничавајући фактор система.

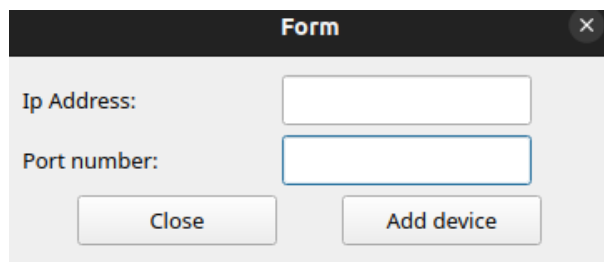
5 Графички кориснички интерфејс

Пошто су мерни подаци стигли до рачунара, преостаје да се покаже како их корисник заправо види и користи. У наставку је описан рад са GUI апликацијом, од повезивања до тумачења резултата.

5.1 Окружење и покретање

За потребе управљања уређајем и визуелизације мерних података у реалном времену коришћена је апликација отвореног кода развијена у оквиру *Open-EPT* пројекта. Апликација ради на рачунару и са уређајем комуницира искључиво преко мреже, о чему је било речи у одељку 2.3.3.

Приликом иницијалног покретања апликације дефинише се радни директоријум (*workspace directory*) у коме се током сесије аутоматски складиште сви генерисани извештаји и сирови подаци измерених величина. Након избора директоријума, апликација се налази у стању чекања на успостављање TCP везе са мрежним модулом уређаја. Након покретања апликације потребно је додати уређај. Кликом на знак + у горњем левом углу отвара се прозор приказан на слици 19.



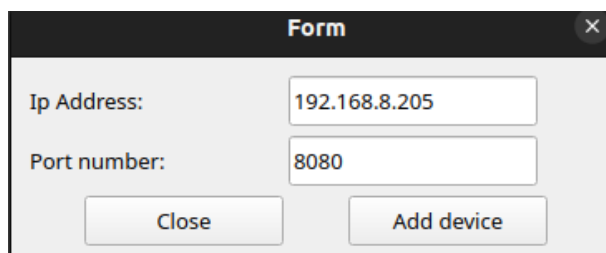
Слика 19: Долазни прозор GUI апликације

Потребно је унети IP адресу и порт на којима ESP плочица одржава TCP сервер. Порт је подразумевано 8080, а IP адреса се може открити командом `ping`:

```
9 technerd@comp stm32cubeide
10 $ ping -c 10 esp_server.local
11 PING esp_server.local (192.168.8.205) 56(84) bytes of data.
12 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=1 ttl=255 time=3.09 ms
13 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=2 ttl=255 time=2.80 ms
14 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=3 ttl=255 time=3.82 ms
15 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=4 ttl=255 time=5.06 ms
16 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=5 ttl=255 time=34.8 ms
17 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=6 ttl=255 time=57.9 ms
18 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=7 ttl=255 time=80.2 ms
19 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=8 ttl=255 time=103 ms
20 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=9 ttl=255 time=5.28 ms
21 64 bytes from ESP-0DEEED (192.168.8.205): icmp_seq=10 ttl=255 time=2.61 ms
22
23 --- esp_server.local ping statistics ---
24 10 packets transmitted, 10 received, 0% packet loss, time 9013ms
25 rtt min/avg/max/mdev = 2.610/29.808/102.528/35.656 ms
```

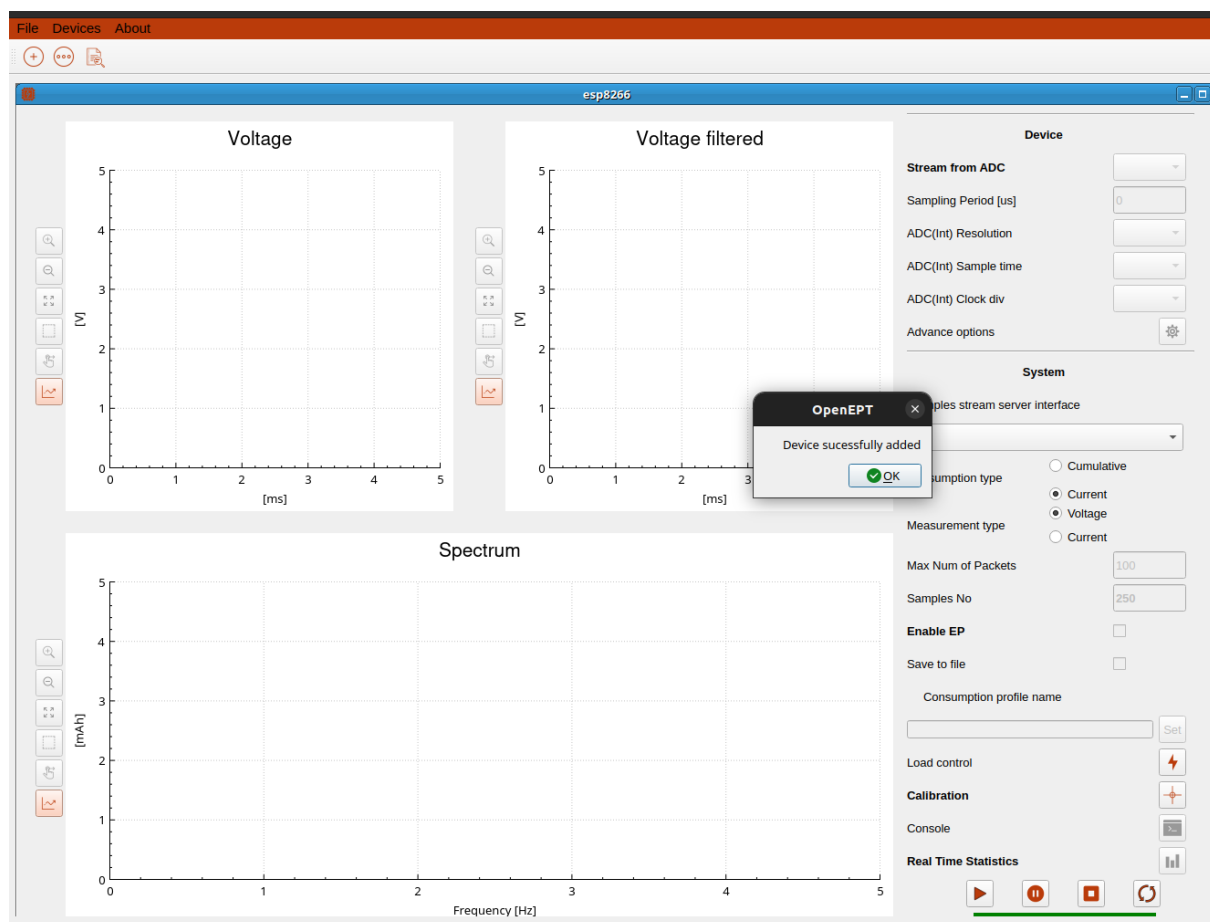
Слика 20: Налажење IP адресе преко `ping` команде

Опција -с 10 означава да се шаље 10 ICMP пакета. На слици се види да одговори стижу са IP адресе 192.168.8.205. Након уношења параметара у долазни прозор добија се приказ на слици 21.



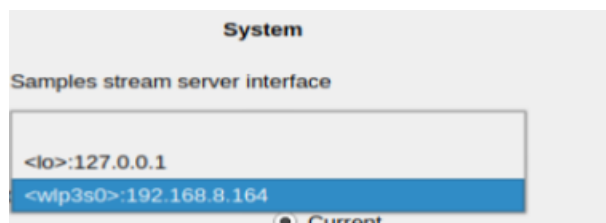
Слика 21: Потребни подаци за повезивање на GUI апликацију

Након овога апликација јавља да је повезивање успешно (слика 22).



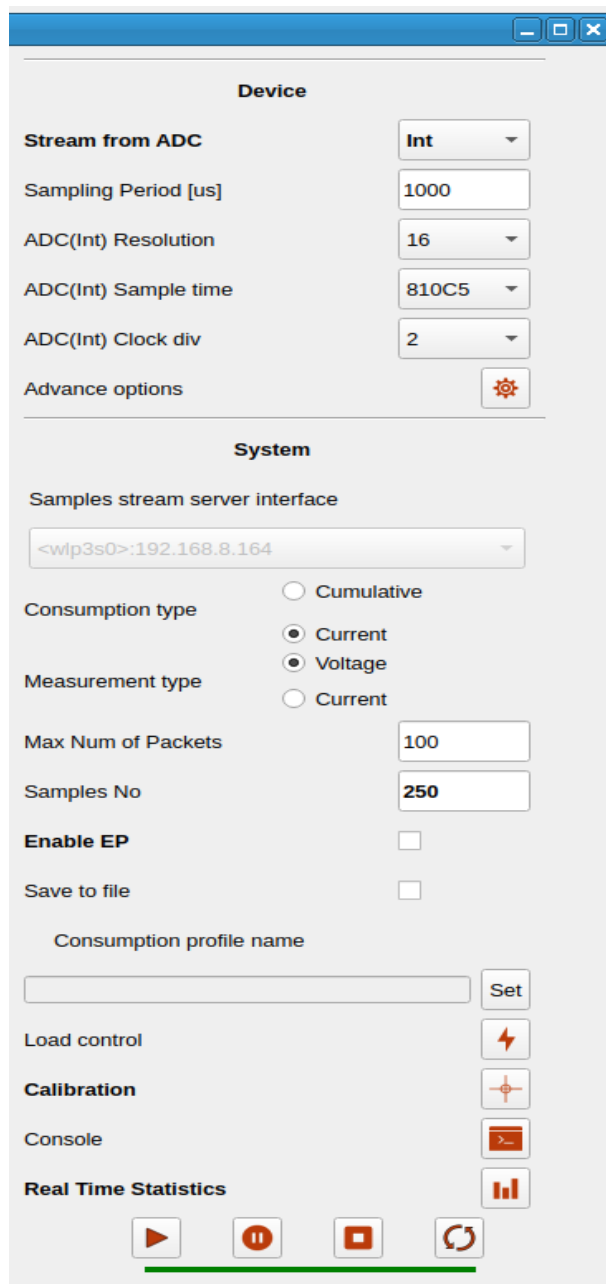
Слика 22: Успешно додавање уређаја

Пре приступа опцијама апликације потребно је одабрати мрежни интерфејс рачунара на који ће уређај слати мерне одбирке (слика 23). У овом случају одабрана је адреса 192.168.8.164, односно адреса рачунара на коју ESP шаље UDP пакете.



Слика 23: Избор IP адресе

Након избора адресе постаје доступна промена параметара у апликацији (слика 24).



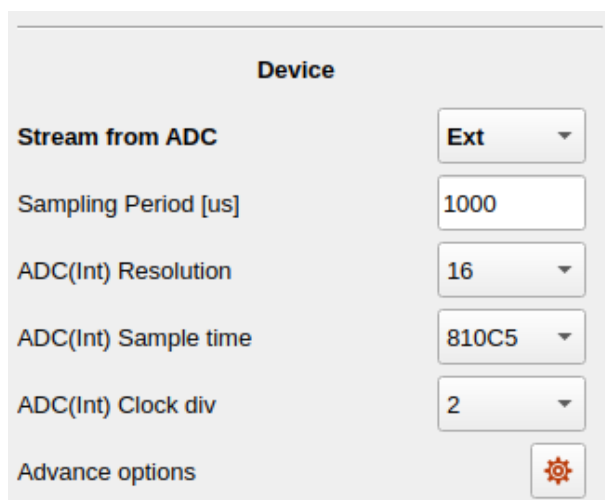
Слика 24: Дозвољена промена параметара

5.2 Кориснички интерфејс и функционалности

Графички кориснички интерфејс подељен је у неколико функционалних целина које омогућавају конфигурисање мерења, начина пријема података и тока аквизиције.

5.2.1 Секција за додавање уређаја

У заглављу интерфејса налазе се опције за дефинисање параметара AD конверзије:



Device	
Stream from ADC	Ext
Sampling Period [us]	1000
ADC(Int) Resolution	16
ADC(Int) Sample time	810C5
ADC(Int) Clock div	2
Advance options	

Слика 25: Секција за додавање уређаја

Опција **Stream from ADC** омогућава избор начина рада AD конвертора. Параметар **Sampling Period [us]** дефинише период одабирања сигнала у микросекундама. Параметар **ADC(Int) Resolution** дефинише резолуцију AD конвертора у битовима, **ADC(Int) Sample time** време одабирања у циклусима (нпр. 810C5), а **ADC(Int) Clock div** делитељ фреквенције такта ADC модула. Дугме **Advance options** омогућава приступ напредним подешавањима хардвера.

5.2.2 Системска секција

Овај сегмент омогућава управљање пристиглим токовима података преко мрежног интерфејса и дефинисање величине пакета (слика 26).

The image shows a software configuration window titled "System". Inside, there is a section "Samples stream server interface" with a dropdown menu showing "<wlp3s0>:192.168.8.164". Below this are two radio button groups: "Consumption type" with "Cumulative" selected, and "Measurement type" with "Current" selected. There are two input fields: "Max Num of Packets" with the value "100" and "Samples No" with the value "250". Below these are two checkboxes: "Enable EP" which is unchecked, and "Save to file" which is checked. At the bottom, there is a text field for "Consumption profile name" containing the text "results", followed by a "Set" button.

Слика 26: System секција

Опција **Samples stream server interface** омогућава одабир мрежног интерфејса и IP адресе са које се врши пријем одбирака (нпр. `wlp3s0:192.168.8.164`). Опција **Measurement type** служи за одабир типа мерења, параметар **Max Num of Packets** дефинише максималан број пакета који се примају у оквиру једне сесије, а **Samples No** број одбирака по пакету. Опције **Enable EP** / **Save to file** служе за активирање додатне обраде и чување измерених података у датотеку, док поље **Consumption profile name** служи за унос назива профила потрошње ради каснијег приступа сачуваним подацима.

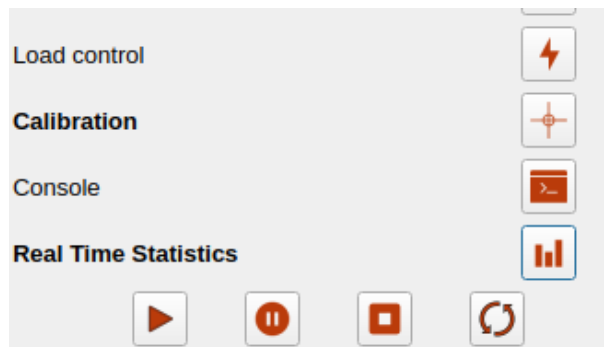
5.2.3 Чување резултата

Као што је приказано у System секцији, постоји опција **Save to file** која омогућава чување резултата аквизиције у фолдер наведен у пољу **Consumption profile name**. У овом случају одабрано је да се резултати чувају у фолдеру `results`, који се појављује унутар претходно изабраног радног директоријума који је у овом случају `~/Documents/GUI_TEST/`

Након постављања одговарајућих подешавања, може се покренути аквизиција. Када се аквизиција заустави, у директоријуму `results` налазе се резултати мерења.

5.2.4 Контрола и напредне алатке

У доњем делу панела налазе се контроле за управљање системом и графичким приказом (слика 27).

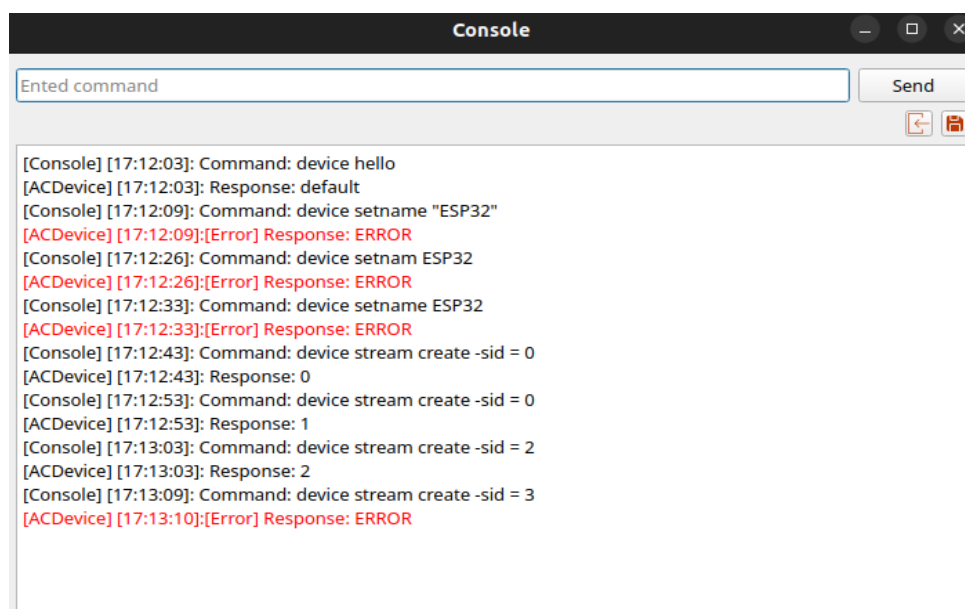


Слика 27: Команде за контролу система

Дугмад **Load control**, **Calibration**, **Console** и **Real Time Statistics** омогућавају брзи приступ контроли оптерећења, калибрацији системских параметара, системској конзоли и статистици у реалном времену, док контролна трака (**Start** / **Pause** / **Stop**) на дну панела омогућава покретање аквизиције, паузирање пријема и заустављање тренутне аквизиције.

Конзола

GUI апликација директно преводи одређене акције (на пример старт / стоп аквизиције) у одговарајуће команде. Међутим, поред овога могуће је исте ове акције текстуално унети у систем преко **конзоле** (слика 28).



Слика 28: Конзола

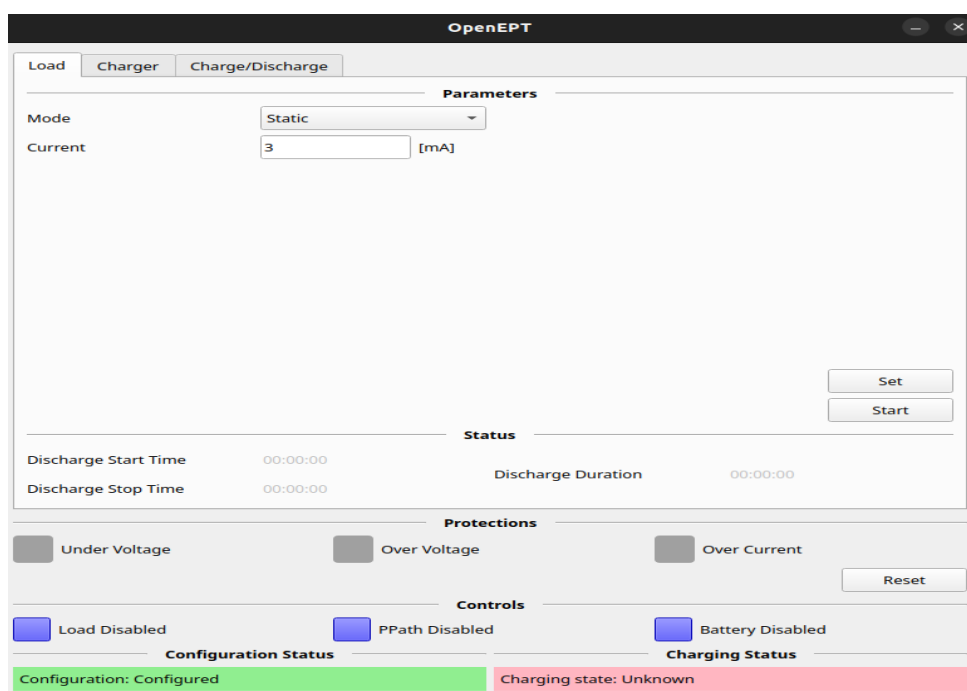
Конзола се састоји од поља за унос команди у систем (командне линије) и сегмента за приказ послатих и примљених порука (дневника команди). У случају да је одговор који ESP микроконтролер прослеђује са STM микроконтролера био успешан (*OK*

/param/), конзола исписује поруку на уобичајени начин (црним словима), при чему је први *OK* параметар изостављен. Са друге стране, уколико из било ког разлога команда није могла бити извршена на STM микроконтролеру, повратна порука се у конзоли исписује црвеним словима.

На приложеном примеру види се да су успешно креирана три тока података, док је четврти одбијен јер систем подржава највише три тока. Ово потврђује да су механизми управљања токовима података, описани у оквиру UDP комуникације, успешно реализовани.

Контрола оптерећења (*Load control*)

Прозор за контролу оптерећења омогућава подешавање константне струје и праћење трајања режима. Хардверске заштите од пренапона, поднапона и прекомерне струје сигнализирани су индикаторима и остају закључане након окидања саме заштите до ручног ресетовања. Такође, омогућено је управљање струјним путањама и праћење стања батерије у реалном времену. Стање батерије се прати кроз поље *Charging Status*, при чему се пражњење региструје када је присутна позитивна вредност струје, пуњење се активира када се детектује негативна вредност струје, док се мировање приказује када је струја приближно једнака нули.



Слика 29: Картица за контролу оптерећења

5.2.5 Дневник догађаја

Дневник догађаја прикупља све битне активности током извршавања програма из два извора: саме апликације и повезаног уређаја. Свака порука се према важности

класификује у један од три нивоа – информативни, упозоравајући или критични — који се ради лакшег препознавања разликују по боји текста и позадине. На тај начин се ефикасно прате стање система и потенцијални проблеми у реалном времену након повезивања уређаја.

```
[Application] [17:43:13]:Sampling time sucessfully obained and presented
[Application] [17:43:14]:Load state sucessfully obained and presented
[Application] [17:43:14]:Load current sucessfully obained and presented
[Application] [17:43:14]:Battery state sucessfully obained and presented
[Application] [17:43:14]:PPath state sucessfully obained and presented
```

Слика 30: Дневник догађаја

5.3 Приказ и тумачење мерних података

Апликација прима одбирке са уређаја, претвара сирове вредности у физичке величине (волте и милиампере) и приказује их у реалном времену. Број одбирака у једном пакету подешава се параметром **Samples No**, при чему су у пакету напон и струја распоређени на један од два начина, у зависности од режима рада.

EXT режим

У **EXT** режиму рада, прва половина пристиглих података у пакету садржи узорке струје, а друга половина узорке напона.

Струја	Напон
$I_0, I_1, \dots, I_{n-1}$	$V_0, V_1, \dots, V_{n-1}$

Табела 4: Структура пакета података у **EXT** режиму рада

Треба напоменути да се у **EXT** режиму на GUI страни над сваким 16-битним податком врши замена редоследа бајтова. (**byte-swap**), чиме се коригује редослед преноса бајтова између микроконтролера и GUI апликације јер STM страна такође врши замену бајтова. Замена се врши на нивоу бита – односно горњи и доњи бити мењају места, чиме виши бајт постаје нижи и обрнуто.

Величина	Бајт 1 (Виши)	Бајт 0 (Нижи)
$V_{received}$	High Byte	Low Byte
$I_{received}$	High Byte	Low Byte

Табела 5: Податак пре замене бајтова

Величина	Бајт 1 (Виши)	Бајт 0 (Нижи)
$V_{converted}$	Low Byte	High Byte
$I_{converted}$	Low Byte	High Byte

Табела 6: Податак након замене бајтова

INT режим

У INT режиму рада, подаци у пакету се шаљу наизменично, почевши од напона, па струје и тако даље.

Индекс узорка (i)	0	1	2	3	...	$2n - 1$
Сирови податак	V_0	I_0	V_1	I_1	...	I_{n-1}

Табела 7: Структура пакета података у INT режиму рада

Претварање у физичке величине

Сирове вредности претварају се у физичке према следећим изразима. Напон у волтима:

$$V = V_{off} + V_{raw} \cdot V_{inc}, \quad V_{inc} = \frac{V_{ref}}{2^N} \cdot k_V \quad (1)$$

где су V_{off} офсет напона, V_{ref} референтни напон конвертора, N број битова конвертора, а k_V фактор скалирања. Струја у милиамперима:

$$I = \frac{I_{raw} \cdot I_{inc} - I_{off}}{R_{shunt} \cdot G} \cdot 1000 \cdot k_I \quad (2)$$

где су I_{off} офсет струје на шант отпорнику, R_{shunt} отпорност шант отпорника, G појачање, а k_I корекциони фактор струје. Кумулативна потрошња у mAh добија се сабирањем струје по формули:

$$C_{cum}[n] = C_{cum}[n - 1] + I[n] \cdot \frac{T_s}{3,6 \times 10^6} \quad (3)$$

где је T_s период одабирања у милисекундама, а фактор 3 600 000 претвара милсекунде у сате.

Параметре за калибрацију могуће је наћи у одељку *System* у графичком корисничком интерфејсу.

The screenshot shows a window titled "Calibration Data" with the following parameters and values:

Parameter	Value	Unit
ADC Voltage Ref	8.179	V
Voltage Offset	0	V
Voltage Correction	1.3348	
Voltage C. Offset	1.6302	V
Current Correction	1	
Current Gain	9.38	
Current Shunt	0.1	ohm

A "Submit" button is located at the bottom right of the window.

Слика 31: Прозор са параметрима за калибрацију

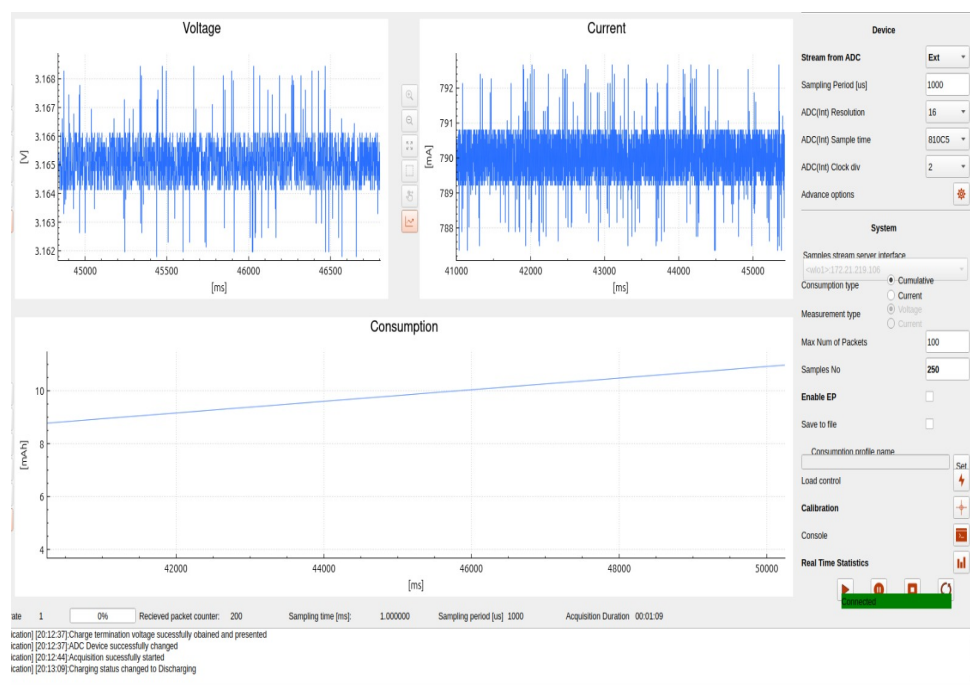
6 Резултати тестирања

У овом поглављу приказани су резултати тестирања реализованог система: најпре је приказан изглед напона и струје са случајно генерисаним подацима, конципираним тако да буду константне вредности са одређеним нивоом шума, након чега су надаље приказани резултати тестирања који су добијени читавањем *INA234* сензора. На крају је описана и статистика пријема коју апликација пружа како би исправност преноса била приказана.

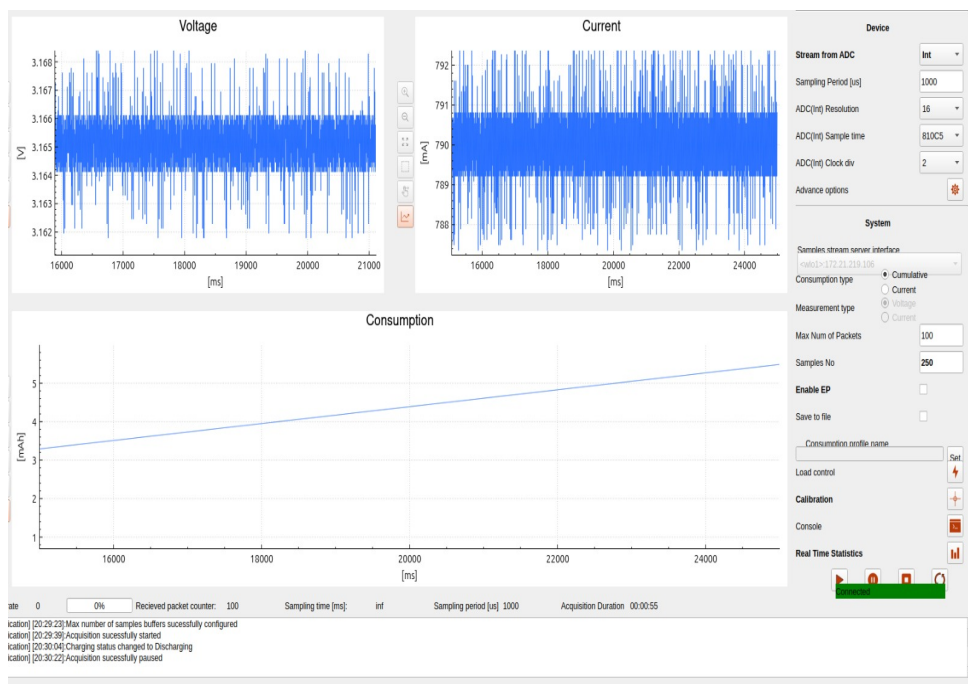
Систем је тестиран у конфигурацији описаној у одељку 3.3, при чему STM микроконтролер генерише податке који се преко ESP-а прослеђују до GUI апликације.

6.1 Приказ резултата

Скалиране вредности приказују се у реалном времену кроз три независна дијаграма: временски дијаграм напона (**Voltage [V]**), временски дијаграм струје (**Current [mA]**) и дијаграм кумулативне потрошње (**Consumption [mAh]**). Поред тога, за сваки пакет се одржавају минимална, максимална и средња вредност напона и струје. Најпре је спроведено систематско тестирање преноса са случајно генерисаним подацима, ради провере исправности у оба режима рада (слике 32 и 33). Може се закључити да оба режима дају вредности за напон и струју у предвиђеним оквирима, што потврђује исправност поступка паковања, преноса и скалирања података.

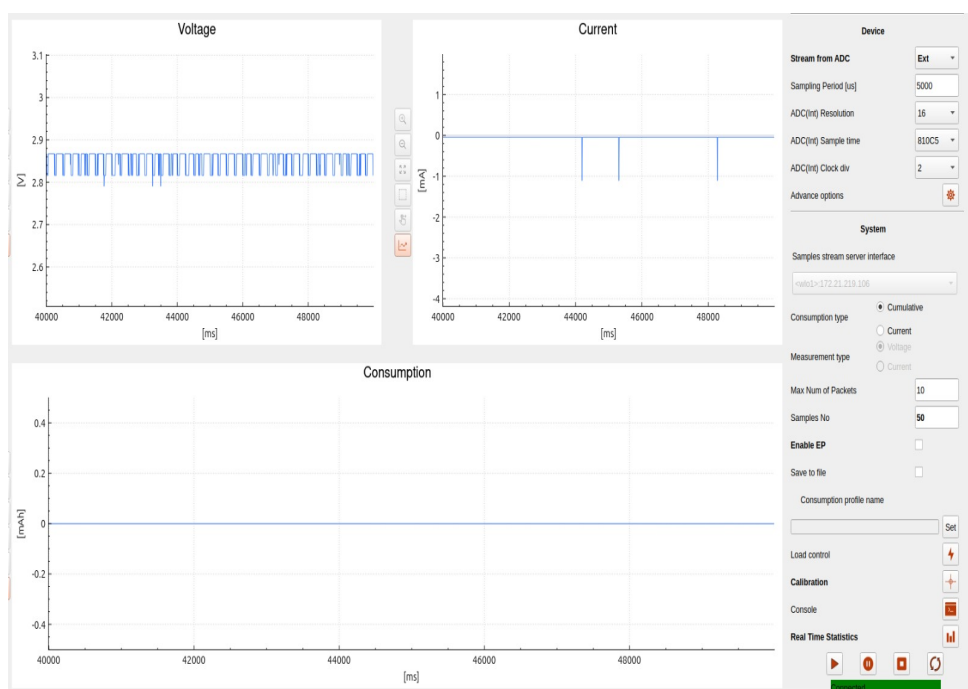


Слика 32: Пренос у EXT режиму са случајно генерисаним подацима



Слика 33: Пренос у INT режиму са случајно генерисаним подацима

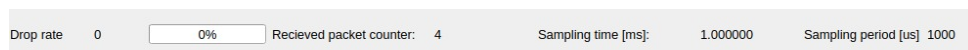
Након тестирања са случајно генерисаним подацима, приказани су и резултати очитавања са сензора INA234 приликом повезивања система у EXT режиму (слика 34). Резултате очитавања сензора има смисла посматрати само у EXT режиму, док у INT режиму нема смисла вршити читање са сензора, с обзиром на то да се у том режиму мерење врши преко интерног аналогно-дигиталног конвертора.



Слика 34: Приказ напона, струје и кумулативне потрошње у EXT режиму

6.2 Статистика пријема

Поред дијаграма, апликација приказује и статистику пријема: број примљених и одбачених пакета, период одабирања и трајање аквизиције (слика 35). Ова статистика омогућава да се одмах провери исправност пренетих података, то јест да ли подаци стижу до GUI апликације, независно од тога како графици изгледају. Одбачени пакети региструју се поређењем бројача пакета у заглављу, о чему је било речи у претходним поглављима.



Слика 35: Приказ статистике пакета

Ослањајући се на ову статистику, тестирање је спроведено при максималном броју одбирака по пакету (250) и различитим периодима одабирања, како би се проверило понашање система у најгорем случају.

С обзиром на то да је циљ теста била провера перформанси преноса података кроз систем, а не понашања самог сензора, коришћени су случајно генерисани (dummy) подаци, чиме је обезбеђен константан и поновљив извор података без утицаја временског одзива сензора на резултате. Идентични резултати би се добили и коришћењем стварних података са сензора (једино не би били тестирани у INT режиму јер то нема смисла него само у EXT), с обзиром на то да брзина преноса зависи од комуникационог канала, а не од извора података. Резултати овог тестирања сумирани су у табели 8.

Режим	Период одабирања	Одбирака по пакету	Одбачени пакети
EXT	1000 μ s	250	0 %
INT	1000 μ s	250	0 %
EXT	5000 μ s	250	0 %
INT	5000 μ s	250	0 %
EXT	10000 μ s	250	0 %
INT	10000 μ s	250	0 %

Табела 8: Резултати тестирања пријема података

7 Закључак

У оквиру овог рада реализован је мрежни слој на ESP микроконтролеру који омогућава пренос података са STM-а до апликације са графичким корисничким интерфејсом, а описано је и коришћење те апликације за управљање преносом и визуелизацију напона и струје. Open-EPT GUI апликација показала се као погодан алат за анализу потрошње енергије: омогућава конфигурацију без поновног програмирања уређаја, приказ напона, струје и кумулативне потрошње у реалном времену, чување профила потрошње и увид у статистику пријема. Тестирањем са генерисаним подацима у оба режима рада (EXT и INT) показано је да оба приступа дају резултате у очекиваном опсегу и да при 250 одбирака по пакету није забележен ниједан изгубљен пакет.

Тестирање је спроведено без стварног спољашњег оптерећења на сензору, услед чега су вредности струје биле блиске нули. Стварна брзина одабирања на STM страни одређена је тајмером од 1 ms, чији се откуцаји бројачем по току података усклађују са параметром **Sampling Period** из GUI-ја, док се читавање сензора у кешу освежава на 10 ms. При периоду одабирања од 1 ms и 250 одбирака по оквиру, оквир се шаље на сваких 250 ms, уз искоришћеност UART везе од свега око 4,4 %. Усклађивање периода освежавања кеша сензора са периодом одабирања остаје као даљи рад. Даљи развој система би стога требало да обухвати тестирање под реалним, динамичким оптерећењем, као и мерење при различитим периодима одабирања и у неповољнијим мрежним условима (слаб WiFi сигнал, прекид везе са GUI-јем). Пошто UDP не пружа гаранцију испоруке, као наредни корак могла би се размотрити и додатна заштита од губитака података у случају захтевнијих мерења.

8 Литература

- [1] OpenEPT GUI Repository, GitHub: <https://github.com/OpenEPT/GUI>
- [2] OpenEPT Firmware Repository, GitHub: <https://github.com/OpenEPT/Firmware>
- [3] OpenEPT CLP Repository, GitHub: <https://github.com/OpenEPT/CLP>
- [4] OpenEPT Organization, GitHub: <https://github.com/OpenEPT>
- [5] I2C Protocol: I2C Protocol explained
- [6] UART protocol: Can UART data bits be increased?
- [7] Texas Instruments, *INA234 Datasheet*, <https://www.ti.com/product/INA234>
- [8] STMicroelectronics, *STM32L476RG Datasheet and Reference Manual*, <https://www.st.com/en/microcontrollers-microprocessors/stm32l476rg.html>
- [9] Espressif Systems, *ESP32-C6-WROOM-1 Datasheet*, <https://www.espressif.com/en/products/socs/esp32-c6>
- [10] FreeRTOS Kernel, <https://github.com/FreeRTOS/FreeRTOS-Kernel>
- [11] Катедра за електронику и дигиталне системе, *Материјали са предавања и вежби из предмета Системи у реалном времену*, Електротехнички факултет, Универзитет у Београду, SRV
- [12] Катедра за рачунарску технику и информатику, *Материјали са предавања из предмета Рачунарске мреже 1*, Електротехнички факултет, Универзитет у Београду
- [13] STMicroelectronics, *STM User Manual (UM1724)*
- [14] J. Postel, *User Datagram Protocol*, RFC 768, 1980, <https://www.rfc-editor.org/rfc/rfc768>
- [15] W. Eddy (ed.), *Transmission Control Protocol (TCP)*, RFC 9293, 2022, <https://www.rfc-editor.org/rfc/rfc9293>
- [16] The Qt Company, *Signals & Slots*, Qt Documentation, <https://doc.qt.io/qt-6/signalsandslots.html>

Списак скраћеница

Скраћеница	Значење
STM	STMicroelectronics микроконтролерска платформа
ESP	Espressif мрежни микроконтролер (ESP8266/ESP32)
GUI	Graphical User Interface — графички кориснички интерфејс
UART	Universal Asynchronous Receiver-Transmitter
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
ADC	Analog-to-Digital Converter — аналого-дигитални конвертор
DAC	Digital-to-Analog Converter — дигитално-аналогни конвертор
I2C	Inter-Integrated Circuit — комуникациони протокол
RTOS	Real-Time Operating System — оперативни систем за рад у реалном времену
HAL	Hardware Abstraction Layer — слој апстракције хардвера
LSB	Least Significant Bit
ISR	Interrupt Service Routine — прекидна рутина
IP	Internet Protocol
HTTP	HyperText Transfer Protocol
MTU	Maximum Transmission Unit
mDNS	Multicast DNS
NTP	Network Time Protocol
SSID	Service Set Identifier

Списак слика

1	Шема повезивања на I2C магистрали	3
2	I2C магистрала и структура једне успешне трансакције	3
3	Структура једног оквира UART пакета	4
4	Положај TCP и UDP протокола у мрежним слојевима	5
5	Open-EPT плоча са STM и ESP микроконтролером	7
6	STM NUCLEO L476RG	8
7	USB To TTL програматор	9
8	Повезивање система са Open-EPT плочом	9
9	FreeRTOS архитектура система	10
10	Садржај TxMsg_t структуре	11
11	Очитавање напона, струје и потрошње са INA234 сензора без повезаног оптерећења	12
12	Дијаграм диспачера команди на ESP-у	14
13	Машина стања бинарног читача (BinaryFrameReader)	14
14	Структура и компоненте мрежног слоја (NetworkLayer)	15
15	Стања једног UDP <i>stream</i> слота	17
16	Ток једне команде: GUI → TCP → ESP → UART → STM и назад	17
17	Ток мерних података: STM → UART → ESP → UDP → GUI	18
18	Структура бинарног пакета.	18
19	Долазни прозор GUI апликације	19
20	Налажење IP адресе преко ring команде	19
21	Потребни подаци за повезивање на GUI апликацију	20
22	Успешно додавање уређаја	20
23	Избор IP адресе	21
24	Дозвољена промена параметара	21
25	Секција за додавање уређаја	22
26	System секција	23
27	Команде за контролу система	24
28	Конзола	24
29	Картица за контролу оптерећења	25
30	Дневник догађаја	26
31	Прозор са параметрима за калибрацију	27
32	Пренос у EХТ режиму са случајно генерисаним подацима	28
33	Пренос у INT режиму са случајно генерисаним подацима	29
34	Приказ напона, струје и кумулативне потрошње у EХТ режиму	29
35	Приказ статистике пакета	30

Списак табела

1	Поређење TCP и UDP протокола и њихова улога у систему	6
2	Акције CommandParser -а за текстуалне поруке са STM-а	15
3	Преглед TCP веза које обрађује ESP	16
4	Структура пакета података у EXT режиму рада	26
5	Податак пре замене бајтова	26
6	Податак након замене бајтова	26
7	Структура пакета података у INT режиму рада	27
8	Резултати тестирања пријема података	30